

APPLE SILICON

From a new laptop to a working LLM program

Eighteen steps, in order, with a command to prove each one worked. Everything you need to install, connect, build and ship your first language-model program on an M-series Mac.

MACHINE

Apple Silicon — M1
or later

PYTHON

3.11, in a virtual
environment

YOU BUILD

A terminal chat
program, twice

TIME

About four hours,
unhurried

How to use this

Work top to bottom, once. Do not skip ahead and do not do two steps at a time — every step ends with a **Verify** line, and the whole document only works if you run it. If a verify fails, the troubleshooting table at the end of that step is where to look before you try anything clever.

You will finish with a folder called `llm-starter` on your Mac and on GitHub, containing a program that talks to a language model two ways: through OpenAI's API over the internet, and through a model running on your own laptop. Everything else in this document exists to make those two files run.

Conventions

YOU SEE	IT MEANS
<code>\$ command</code>	Type this into Terminal. Do not type the <code>\$</code> .
<code>>>> code</code>	Type this inside a running Python session.
<code><your-name></code>	Replace the whole thing, brackets and all, with your own value.
Verify	A command that proves the step worked. Never move on without it.
No admin	The alternative when you cannot type an administrator password.
Rule	Something worth remembering after this document is closed.

The five phases

PREPARE — STEPS 01-03

- 01 The terminal, and the twelve commands you need
- 02 Files, paths, and PATH itself
- 03 What your Mac already has

INSTALL — STEPS 04-08

- 04 Python 3.11
- 05 The project folder and its virtual environment
- 06 Installing packages with pip
- 07 The other three: conda, uv, poetry
- 08 VS Code

CONNECT — STEPS 09-13

- 09 Git, and a `.gitignore` before anything else
- 10 A GitHub account and an SSH key
- 11 A Hugging Face token

- 12 API keys: OpenAI, Anthropic, Gemini
- 13 `.env`, and what to do when a key leaks

BUILD — STEPS 14-16

- 14 `chat_openai.py` — a model over the internet
- 15 `chat_local.py` — a model on your laptop
- 16 Freezing `requirements.txt`

SHIP — STEPS 17-18

- 17 Docker, one image, both scripts
- 18 Pushing to GitHub

APPENDICES

- A The day-one script
- B Printable checklist
- C Glossary

Two things before you start. Give yourself an unhurried afternoon rather than twenty minutes between meetings — most of the pain in setup comes from rushing a step and carrying the breakage forward. And if a command asks for a password you do not have, look for the **No admin** box in that step; there is one everywhere it matters.

01

PREPARE

The terminal, and the twelve commands you need

The terminal is a program that reads a line of text, runs it, and prints what happened. That is the whole idea. What makes it intimidating is the blank prompt, which gives you no hint about what it will accept — so here are the twelve commands that cover almost everything in this document, and the four habits that make typing them bearable.

Opening it. Press Cmd-Space, type “Terminal”, press Return. You will get a window with a cursor waiting. Nothing you type here can break your laptop unless you tell it to delete something, and this document never asks you to.

Where am I?

```
$ pwd                # print working directory — where you are now
$ ls -la             # list everything here, hidden files included
$ cd ~               # go home; ~ means /Users/you
```

Listing 1.1 — Run all three now. The output of `pwd` should be your home directory.

Those three answer the question that causes more confusion than any other: *which folder am I actually in?* Almost every “file not found” is really “I am not where I think I am”. Run `pwd` whenever something surprises you.

The twelve

COMMAND	WHAT IT DOES
<code>pwd</code>	Where am I
<code>ls -la</code>	What is here, hidden files and all
<code>cd folder</code>	Go there. <code>cd ..</code> up one level, <code>cd ~</code> home, <code>cd -</code> back where you were
<code>mkdir -p a/b</code>	Make a folder, and any parent folders it needs
<code>cp a b</code>	Copy a file. <code>cp -r</code> for a whole folder
<code>mv a b</code>	Move a file, or rename it — the same operation
<code>rm file</code>	Delete. There is no undo and no wastebasket. Read twice.
<code>cat file</code>	Print a whole file to the screen
<code>which -a python3</code>	Which program will actually run when you type that name
<code>grep -rn "text" .</code>	Find text inside files, recursively, with line numbers
<code>df -h .</code>	How much disk space is left
<code>history</code>	What did I just run

The four habits

Tab completes. Type three letters of a folder name and press Tab. This is not a shortcut — it is how you stop mistyping paths, which is the single most common cause of “command not found”.

Up-arrow recalls. The previous command, and the one before that. Ctrl-R searches your history. After a week you will retype almost nothing.

Ctrl-C stops. Whatever is running, Ctrl-C ends it. If the terminal seems frozen, it is usually a program waiting for input — Ctrl-C first, then Ctrl-D.

Copy carefully. If you paste a command from a document or web page and it fails with a strange character error, the quotes were “smart quotes”. Retype them by hand. This costs someone an hour every single week.

RULE 01

Change one thing, then check it. Two changes at once means you no longer know which one worked — and finding out takes longer than doing it properly did.

Never paste a command you do not understand, especially one that starts `curl` and ends `| bash`. Ask what it does first. This is not paranoia — it is the one irreversible mistake available to a beginner.

When it goes wrong

SYMPTOM	CAUSE	FIX
command not found	A typo, or the program is not installed.	Tab-complete the name. If it still fails, it is not installed yet — that is what the Install phase is for.
No such file or directory for a file you can see	You are in a different folder, or the path has a space in it.	pwd , then ls . Quote paths with spaces: cd "My Folder" .
Permission denied	The file is not executable, or it is not yours.	ls -l to see. Step 02 explains the letters.
The terminal ignores everything I type	A program is waiting for input, or Ctrl-S froze the output.	Ctrl-C to abort, Ctrl-Q to unfreeze, Ctrl-D to say "no more input".
A pasted command fails on a quote character	Smart quotes from a document.	Retype the quotes yourself.

DO THIS NOW

1. Run pwd , then ls -la , and read the output. You should see folders like Documents and Downloads , plus some files starting with a dot.
2. Make a folder: mkdir -p ~/projects . Then cd ~/projects and run pwd to confirm you moved.
3. Run df -h . and write down how much space is free. You will want at least 20 GB for this document, and 50 GB is comfortable.

Verify — you can move between folders and list their contents without touching a mouse.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Two places, every cohort. First, people type the `$` . Say out loud that the dollar sign is the prompt, not part of the command. Second, people do not realise the terminal has a “current folder” at all, so they run commands from home and wonder why the project files are missing. Make them run `pwd` before every command for the first ten minutes — it feels tedious and it fixes the mental model permanently.

02 PREPARE Files, paths, and PATH itself

Two different things are called “path”, and mixing them up produces a very specific bewilderment. A *file path* is where something lives. `PATH`, in capitals, is a list of folders your shell searches when you type a command name. Nearly every “I installed it but the computer says it does not exist” is a `PATH` problem, and you are going to meet one in the Install phase.

File paths

An absolute path starts at the top and is unambiguous: `/Users/you/projects/llm-starter`. A relative path starts wherever you are: `./data` means “the data folder here”, `../` means “one level up”. `~` is shorthand for your home folder. Use relative paths inside a project so it still works on someone else's machine.

Hidden files, which is most of what matters here

A filename starting with a dot is hidden from ordinary listings. Not secret — just quiet. You are about to create four of them: `.venv` (your Python environment), `.env` (your secret keys), `.gitignore` (what git should skip) and `.git` (the repository itself). `ls -la` shows them; get in the habit of the `-a`.

Permissions, in one paragraph

```
$ ls -l notes.txt
-rw-r--r--  1 you  staff   2048 May 17 09:12 notes.txt
|   |   |
|   |   └─ everyone else can read
|   └─── your group can read
└────── you can read and write
```

Listing 2.1 — Ten characters: type, then read/write/execute for you, your group, and everyone.

Two commands matter. `chmod +x script.sh` makes a file runnable. `chmod 600 .env` means only you can read it — do that to every file containing a secret, which you will do for real in step 13.

PATH: the list that decides what runs

When you type `python3`, the shell walks through `PATH` from left to right and runs the *first* match it finds. Left to right is the whole game. It is why activating a virtual environment changes which Python you get — activating simply puts a folder at the front of the list — and why an old line in your shell configuration can silently hide a fresh install.

```
$ echo $PATH | tr ':' '\n'      # one folder per line, in search order
$ which -a python3           # every match, not just the winner
```

Listing 2.2 — `which -a` is the most useful diagnostic command in this document. Remember it.

Changing PATH

Your shell reads a configuration file when it starts. On this machine that file is `~/.zshrc`. To put a folder at the front of your PATH permanently, add one line to it:

```
$ echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
$ source ~/.zshrc                # apply it to THIS terminal too
$ echo $PATH | tr ':' '\n' | head -3
```

That `source` line matters. Editing the file changes every *future* terminal; it does nothing to the window you are already in. Everyone trips over this once — usually while wondering why a freshly installed tool is still “not found”.

RULE 02

When a tool behaves impossibly, run `which -a <tool>`. You are almost certainly running a different copy from the one you think you installed.

When it goes wrong

SYMPTOM	CAUSE	FIX
Installed something, shell still says <code>command not found</code>	Its folder is not on PATH, or you have not reloaded the shell.	Add the export line above, then <code>source</code> it or open a new terminal.
Two versions of a tool and the wrong one wins	An earlier PATH entry is hiding the later one.	<code>which -a tool</code> , then put the one you want first. Do not delete the other.
Works in the terminal, fails in VS Code	VS Code started a different kind of shell and read a different file.	Also put the line in <code>~/.profile</code> . And in step 08, select the interpreter explicitly.
<code>PATH</code> looks enormous or repeated	The same export line is in two configuration files, running twice.	<code>grep -n PATH ~/.zshrc ~/.profile</code> and keep one.
<code>Permission denied</code> running my own script	Missing the execute bit.	<code>chmod +x script.sh</code>

DO THIS NOW

1. Print your PATH one folder per line. Count the entries.
2. Run `which -a python3`. Note every path it prints — you will compare against this in step 04.
3. Create `~/.local/bin` and add it to the front of your PATH using the two commands above.
4. Confirm with `echo $PATH | head -c 60` that your new folder is first.

Verify — `echo $PATH` starts with your `.local/bin` folder, in a terminal you opened before you made the change.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

The `source` step. Someone edits the file, opens no new terminal, and concludes the instructions are wrong. Demonstrate it live: add a line, run the command, watch it fail, run `source`, watch it work. Also expect one person to have written `>` instead of `>>` and truncated their shell configuration — mention the difference before they type it, not after.

03 PREPARE What your Mac already has

Your Mac ships with more of this than you might expect, and one thing you must not touch. Five minutes here saves an hour later.

Take an inventory

```
$ uname -m                # arm64 = Apple Silicon. Expected.
$ sw_vers                 # macOS version
$ sysctl hw.memsize | awk '{print $2/1073741824" GB RAM}'
$ df -h / | tail -1       # free disk
$ python3 --version       # Apple's Python. Note it; do not use it.
$ git --version           # may trigger a tools prompt – accept it
```

Listing 3.1 — Write these six answers down. Every support conversation for the next year starts with them.

If `uname -m` says anything other than `arm64`, you are not on Apple Silicon and this is the wrong document for you. If `git --version` pops up a dialogue offering to install command line developer tools, accept it — that single dialogue installs git, a compiler and several things you would otherwise chase individually. It needs no administrator password.

The Python you must not use

`/usr/bin/python3` belongs to Apple. Parts of macOS use it. Never install project packages into it, and never delete it. Modern macOS will actually refuse the install with a message about an “externally managed environment”, which is the operating system protecting itself and you. Step 04 installs your own Python instead.

RULE 03

Never install packages into the Python that came with your operating system. Not once, not “just to test”. It is the fastest route to a laptop that needs reinstalling.

Homebrew — useful, optional, needs a password

Homebrew is the package manager most Mac developers use. It makes step 04 a one-liner. It also asks for an administrator password during installation, so if you do not have one, skip it — step 04 has a complete alternative that does not.

```
$ /bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
$ # then follow its final instructions to add brew to your PATH – it prints them
$ brew --version
```

No admin. Skip Homebrew entirely. In step 04 use the `uv` route, which installs a private Python 3.11 into your home folder with no password and no compiler.

What your Mac can and cannot do

THIS DOCUMENT'S WORK	ON YOUR MACHINE
Everything in Prepare, Install and Connect	Yes, comfortably
<code>chat_openai.py</code> — a model over the internet	Yes. The model runs on someone else's hardware.
<code>chat_local.py</code> — a 1.5B model on your laptop	Yes, using Apple's <code>mps</code> acceleration. Wants 16 GB of RAM to be pleasant; works on 8 GB.
Docker (step 17)	Yes, but Docker Desktop needs an administrator password to install.
GPU inside a Docker container	No. Apple Silicon has no GPU passthrough to Linux containers. Containers here are CPU-only, which is fine for what we build.
Training or fine-tuning a large model	Not on a laptop. That is a rented-GPU job.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>xcrun: error: invalid active developer path</code>	Command line tools missing or broken after an OS upgrade.	<code>xcode-select --install</code> , accept the dialogue, then reopen the terminal.
<code>uname -m</code> says <code>x86_64</code> on a new Mac	You are in a Rosetta terminal session.	Right-click Terminal in Applications → Get Info → untick "Open using Rosetta", then reopen.
<code>brew: command not found</code> right after installing it	The installer told you to add it to PATH and you did not.	Scroll up in the terminal, find its "Next steps", run those two lines. Step 02 explains why.
Homebrew asks for a password you do not have	Managed laptop.	Do not fight it. Use the <code>uv</code> route in step 04.
<code>pip install</code> says <code>externally-managed-environment</code>	You are installing into Apple's Python.	Correct behaviour. Step 05 makes you a virtual environment.

DO THIS NOW

1. Run all six inventory commands and write the answers on paper.
2. Accept the developer tools dialogue if it appears, and wait for it to finish before continuing.
3. Decide your route: *with* an administrator password, install Homebrew now; *without*, skip it and note that you are taking the `uv` path in step 04.

Verify — `uname -m` prints `arm64`, `git --version` prints a version, and you know how much free disk you have.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

The developer tools dialogue appears *behind* the terminal window for some people, so they sit watching a hung `git --version`. Tell the room to check their other windows. Also: at least one person will have 8 GB of RAM and will find step 15 slow — flag it now so it reads as expected rather than broken.

04 INSTALL Python 3.11

Your machine does not have “Python” on it. It has several: one from the operating system, possibly one from an installer you ran last year, and shortly one of your own — plus one inside every virtual environment you create. They do not know about each other. Knowing which one you are talking to at any moment is the single most useful skill in this phase.

Why 3.11 specifically

Not because it is the newest — it is not. The machine-learning ecosystem lags the language. Packages like PyTorch and numpy ship pre-built files for a specific set of Python versions, and being one version ahead of that set means waiting for a compiler to fail on your laptop instead of downloading a working binary in ten seconds. One or two versions behind the newest release is the sweet spot, permanently. When 3.11 is genuinely old, apply the same reasoning rather than the same number.

Route A — Homebrew, if you installed it in step 03

```
$ brew install python@3.11
$ python3.11 --version          # expect 3.11.x
$ which -a python3.11
```

Route B — uv, no administrator password needed

`uv` is a single small program that can download a self-contained Python for you. No compiler, no password, nothing installed outside your home folder. Even if you took Route A, installing `uv` is worth the ten seconds — step 07 uses it.

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
$ source ~/.zshrc          # or open a new terminal
$ uv --version
$ uv python install 3.11
$ uv python find 3.11      # where it put it
```

Listing 4.1 — Read that install script before piping it to a shell if you can. Step 01 said so and meant it.

Confirm which Python is which

```
$ which -a python3 python3.11
$ python3.11 -c "import sys; print(sys.executable); print(sys.version)"
```

`sys.executable` is the truth. It reports the interpreter actually running your code, whatever the shell claims. When something impossible is happening in step 14, this is the command that explains it.

RULE 04

Call the version explicitly — `python3.11`, not `python` — whenever you are creating something. Ambiguity is free to avoid and expensive to debug.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>python3.11: command not found</code> after installing	Not on PATH, or shell not reloaded.	Step 02. <code>source ~/.zshrc</code> or open a new terminal, then <code>which -a python3.11</code> .
<code>brew install</code> fails asking for a password	Managed laptop.	Route B. It needs nothing.
Homebrew upgraded Python later and things broke	Environments link to the interpreter, which moved.	Delete <code>.venv</code> and rebuild it from <code>requirements.txt</code> — step 16 makes that a one-liner.
<code>python --version</code> says 2.7 or nothing at all	Unversioned <code>python</code> barely exists on modern macOS.	Normal. Use <code>python3.11</code> , or work inside the environment from step 05 where plain <code>python</code> exists.
<code>uv</code> install fails with a certificate error	Corporate network inspecting traffic.	You need your organisation's root certificate installed. Ask IT; this is a routine request.

DO THIS NOW

1. Install Python 3.11 by Route A or Route B.
2. Run `python3.11 --version` and confirm it prints 3.11.something.
3. Run `which -a python3.11` and note the path. Compare it with the list you wrote in step 02.

Verify — `python3.11 -c "import sys; print(sys.executable)"` prints a path that is *not* `/usr/bin/python3`.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

People run Route A, see the Homebrew output scroll past, and miss that `python3.11` is the command — they keep typing `python`. Write both on the board. Also, anyone on a corporate Mac will hit the certificate error on the `uv` installer; have the root certificate ready before the session rather than debugging it live.

05 INSTALL The project folder and its virtual environment

Now you make the thing this whole document is about. A folder, and a Python environment inside it that belongs to it alone.

What a virtual environment actually is

It is a folder. Inside it sits a link to a Python interpreter and a place to put packages, and nothing else on your machine knows or cares that it exists. Activating it does one modest thing: it puts that folder's `bin` directory at the front of your `PATH`. That is the entire trick — and it is exactly the mechanism from step 02, which is why that step came first.

Two consequences worth holding on to. An environment is **disposable**: if it misbehaves for more than ten minutes, delete the folder and rebuild it, which costs seconds once step 16 is done. And it is **not portable**: it contains absolute paths, so never move or copy one — rebuild instead.

Make the project

```
$ mkdir -p ~/projects/llm-starter
$ cd ~/projects/llm-starter
$ pwd                                # confirm you are inside it
```

Everything from here to the end of the document happens inside that folder. If a command fails oddly, `pwd` first.

Create and activate

```
$ python3.11 -m venv .venv
$ source .venv/bin/activate
$ which python                    # must point INSIDE .venv
$ python --version                 # plain 'python' now works, and is 3.11
$ deactivate                       # leaves it; try this, then activate again
```

Listing 5.1 — Four operations: create, activate, check, leave. There are no others.

Your prompt now shows `(.venv)` when the environment is active. That marker is your friend — glance at it before every install command for the rest of this document.

Why it is called `.venv`

Convention, and a useful one. VS Code finds a folder with that name automatically (step 08), every `.gitignore` template already excludes it, and every tool in step 07 expects it. Keep it inside the project, keep the name, and never commit it — it holds thousands of machine-

specific files.

RULE 05

One project, one environment, one Python version. Activate before you install anything, and check the `(.venv)` in your prompt if you are not sure.

The alternative to activating

Activation is a convenience for interactive work. Two habits avoid it entirely and are more reliable inside editors and scripts: call the interpreter by its full path — `.venv/bin/python script.py` — or use `uv run script.py`, which resolves the environment itself. Both are immune to the commonest confusion here, which is an environment activated in one terminal tab and not another.

When it goes wrong

SYMPTOM	CAUSE	FIX
No module named venv	On Debian and Ubuntu the venv piece is a separate package.	<code>sudo apt install python3.11-venv</code> , then delete any half-made <code>.venv</code> and try again.
<code>Activate.ps1</code> cannot be loaded because running scripts is disabled	PowerShell execution policy.	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> . No admin needed.
Activated, but <code>python</code> is still the system one	Activation did not take, or you are in a different tab.	Check for <code>(.venv)</code> in the prompt; run the which-command above; activate again.
I moved the project folder and nothing works	Paths inside an environment are absolute.	Delete <code>.venv</code> and recreate it. Never move an environment.
Two environments seem active at once	You activated one inside another.	Deactivate until the prompt is clean, then activate one.
<code>.venv</code> is enormous	Normal — thousands of files.	That is why it is never committed. Step 09 adds it to <code>.gitignore</code> .

DO THIS NOW

1. Create `~/projects/llm-starter` and `cd` into it.
2. Create `.venv` with Python 3.11, activate it, and confirm the interpreter path points inside the project.
3. Deactivate. Confirm the path changes back. Activate again.
4. Delete `.venv` entirely and recreate it, to prove to yourself that it costs nothing.

Verify — `python -c "import sys; print(sys.prefix)"` prints a path inside `llm-starter`.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Activation, in three ways. People open a second terminal tab and lose the environment without noticing. People run the create command from home rather than the project folder. And people on Windows try the Linux activation line, or vice versa. The prompt marker is the cure for all three — make “look at your prompt” the reflex before any install.

06 INSTALL Installing packages with pip

pip fetches packages from the Python Package Index and installs them into whichever interpreter runs it. That last clause is where all the trouble lives, and it is why every command in this document says `python -m pip` rather than plain `pip`.

The one habit that prevents half of all problems

```
$ python -m pip install requests      # right: installs into the Python you are
running
$ pip install requests                # risky: installs into whichever pip won the
PATH race
```

RULE 06

Write `python -m pip`, never bare `pip`. The long form installs into the interpreter you are actually using; the short form is a guess.

Install what this project needs

Make sure your environment is active — check for `(.venv)` — then:

```
$ python -m pip install --upgrade pip
$ python -m pip install openai python-dotenv
$ python -m pip list
```

Listing 6.1 — Two packages. `openai` talks to the API in step 14; `python-dotenv` loads your keys from a file in step 13.

Look at that `pip list` output. You asked for two packages and got eight or nine. The extras are dependencies — things `openai` needs. That is normal, and step 16 is about writing down exactly which versions you ended up with.

The commands worth knowing

COMMAND	WHAT IT DOES
<code>python -m pip install X</code>	Install the newest compatible version
<code>python -m pip install "openai==1.40.0"</code>	Install exactly that version
<code>python -m pip install -r requirements.txt</code>	Install everything a project declares
<code>python -m pip uninstall X</code>	Remove it — but not the things it dragged in
<code>python -m pip list</code>	What is installed here
<code>python -m pip list --outdated</code>	What has newer releases
<code>python -m pip show -f openai</code>	Version, install location, dependencies, files
<code>python -m pip check</code>	Report packages whose dependencies are broken
<code>python -m pip freeze</code>	Exactly what is installed, in a form you can replay — step 16
<code>python -m pip cache purge</code>	Reclaim disk from downloaded package files

Reading a version specifier

WRITTEN AS	MEANS
<code>==1.40.0</code>	Exactly this. What <code>freeze</code> produces.
<code>>=1.40</code>	This or newer. Fine while developing, risky unpinned later.
<code>>=1.40,<2</code>	The sane default: fixes and features, no breaking changes.
<code>~=1.40.0</code>	1.40.x only — patch updates.

When it goes wrong

SYMPTOM	CAUSE	FIX
externally-managed-environment	You are installing into the system Python — the environment is not active.	Activate <code>.venv</code> and try again. Never use <code>--break-system-packages</code> .
Defaulting to user installation because normal site-packages is not writeable	No environment active.	Activate. This warning means your install is not going where you think.
Installed fine, but <code>import</code> fails	It went into a different interpreter.	<code>python -m pip show openai</code> and compare its Location with <code>sys.path</code> .
Could not build wheels for X, pages of red	No pre-built file for your Python version or platform.	Do not install a compiler. Check the package supports 3.11 — this is why step 04 pinned it.
ResolutionImpossible	Two requirements genuinely conflict.	Read only the lines naming two packages and a version. Relax the one you control.
SSL: CERTIFICATE_VERIFY_FAILED	Corporate network inspecting traffic.	Install your organisation's root certificate. Do not disable verification.
Everything reinstalls from scratch each time	Cache purged or unavailable.	Normal after <code>cache purge</code> . Otherwise check disk space.

DO THIS NOW

1. With `.venv` active, upgrade pip, then install `openai` and `python-dotenv`.
2. Run `python -m pip list` and count the packages. Pick two you did not ask for and use `pip show` to find out who needed them.
3. Run `python -m pip check` and confirm it reports nothing.

Verify — `python -c "import openai, dotenv; print(openai.__version__)"` prints a version rather than an error.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Installing without the environment active, then wondering why the import fails — or worse, succeeding globally and breaking a later step in a way that looks unrelated. Two defences: insist on `' + pyM + ' -m pip`, and make “is `(.venv)` in your prompt?” the first question you ask for any install problem all afternoon. It will be the answer more often than anything else.

07

INSTALL

The other three: conda, uv, poetry

You have pip working, and pip is what this document uses. But you will meet three other tools in tutorials, job descriptions and colleagues' repositories, so here is what each one is for, what it is genuinely better at, and how to recognise which one a project is using. You do not have to install any of them to finish this document — though `uv` earns its ten seconds.

The four, side by side

	VENV + PIP	CONDA	UV	POETRY
Installs Python itself	No	Yes	Yes	No
Non-Python things (compilers, FFmpeg, CUDA)	No	Yes	No	No
Speed	Adequate	Slow	Very fast	Slow to resolve
Records exact versions	Via <code>freeze</code>	Weakly	<code>uv.lock</code>	<code>poetry.lock</code>
Present on every machine	Yes	No	No	No
Best at	Universality	Science and GPU stacks	Everyday speed	Publishing libraries

This document recommends **venv + pip** for one reason: it is present everywhere, every tutorial assumes it, and when something breaks the error you have to read is pip's. Learn it first even if you end up using something else daily.

conda — when you need more than Python

conda's trick is that it installs non-Python things too: compilers, system libraries like FFmpeg, CUDA runtimes, and the interpreter itself. In scientific and GPU work that matters, because the hardest installs are often not Python at all. Install **miniforge**, not full Anaconda — minimal, community-run, and free of Anaconda's commercial licensing questions.

```
$ conda create -n aie python=3.11
$ conda activate aie
$ conda install -c conda-forge numpy jupyterlab
$ conda env export --from-history > environment.yml # what you ASKED for —
portable
$ conda deactivate
```

Two cautions. It is slow, because the problem it solves is genuinely hard. And do not mix conda and pip carelessly in one environment — install everything you can with conda first and pip last, because conda cannot see what pip did afterwards.

uv — the same job, much faster

Every pip command you know works with `uv pip` in front, against the same ordinary `.venv`. Installs that took a minute take a second or two, which changes how you work: rebuilding an environment stops being a decision and becomes a reflex.

```
$ uv venv --python 3.11 # creates .venv, downloading 3.11 if needed
$ uv pip install openai python-dotenv
$ uv pip freeze > requirements.txt
$ uv run python chat_openai.py # runs it without activating anything
```

That last line is worth noticing. `uv run` resolves the environment itself, which sidesteps the entire class of “I forgot to activate” problem.

poetry — for libraries you publish

poetry treats your work as a package rather than a folder of scripts: it separates production from development dependencies, records exact versions, and can build and publish a distribution. If you are writing something other people will install, this is the discipline you want. For a chat script it is more ceremony than the job needs.

```
$ poetry init # interactive; writes pyproject.toml
$ poetry add openai python-dotenv
$ poetry add --group dev pytest
$ poetry install # from poetry.lock
$ poetry show --tree # the best "why is this here?" of any tool
```

How to tell what a repository uses

IF YOU SEE THIS FILE	RUN THIS
requirements.txt	python -m pip install -r requirements.txt
environment.yml	conda env create -f environment.yml
uv.lock	uv sync
poetry.lock	poetry install
pyproject.toml and no lock file	python -m pip install -e .

RULE 07

One tool per project, named in the README. Two of them managing the same folder is how a team loses a day.

When it goes wrong

SYMPTOM	CAUSE	FIX
conda stuck on “Solving environment” for minutes	The old solver, or too many loose constraints.	Pin Python, use only conda-forge. Modern conda uses the fast solver by default.
(base) permanently in my prompt after installing conda	Auto-activation.	conda config --set auto_activate_base false
A conda environment broke after a pip install	pip overwrote a conda-managed package.	Rebuild from environment.yml . conda first, pip last, always.
uv: command not found	Its folder is not on PATH.	Step 02, then a new terminal.
poetry put the environment somewhere surprising	Its default is a central cache, not the project.	poetry config virtualenvs.in-project true , then recreate.
A repository has both requirements.txt and poetry.lock	Half-finished migration.	Ask which is current. Do not run both.

DO THIS NOW

1. Install `uv` if you have not already, and time `uv venv` against `python -m venv` on a throwaway folder. The difference is the point.
2. Do *not* convert `llm-starter` to another tool. It stays on `venv` + `pip` for the rest of this document.
3. Write the four filenames from the table above on the inside cover of your notes. You will need to recognise them within a fortnight.

Verify — you can look at an unfamiliar repository, name which tool it uses, and give the one command that sets it up.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Enthusiasm. Someone reads about `uv`, converts `llm-starter` mid-document, and then every subsequent command in the handout is subtly wrong for them. Say explicitly: try the other tools on a scratch folder, never on the project. The other risk is conda's installer changing their shell configuration and quietly hijacking `python` — if a later step goes strange for someone who installed conda, check `which -a python` first.

08 INSTALL VS Code

An editor is not where the difficulty of this work lives, so the goal here is to get VS Code out of your way in twenty minutes. Two things matter: that it is using the same Python as your terminal, and that you know where the terminal panel is.

Install

Download it from code.visualstudio.com, open the `.zip`, and drag **Visual Studio Code** into your Applications folder. No administrator password needed.

Then add the `code` command so you can open folders from the terminal: command palette (Cmd-Shift-P) → *Shell Command: Install 'code' command in PATH*.

```
$ code --version
$ cd ~/projects/llm-starter && code . # opens this folder
```

The extensions worth having

Python (the official Microsoft one — it brings the language support), and **Jupyter** if you will use notebooks later. That is it for now. Resist installing twenty more; each is a background process and a source of confusing behaviour, and an editor that starts in two seconds instead of eight is a tax you avoid paying all day.

The one thing to get right: the interpreter

Open the command palette with **Cmd/Ctrl-Shift-P**, type *Python: Select Interpreter*, and choose the one inside `./.venv`. The chosen path appears in the status bar at the bottom of the window. Check it whenever imports behave strangely.

```
$ # the path you are looking for in the picker:
$ echo "$PWD/.venv/bin/python"
```

Then commit the setting so the choice travels with the project. Create `.vscode/settings.json`:

```
{
  "python.terminal.activateEnvironment": true,
  "editor.formatOnSave": false,
  "files.exclude": { "**/__pycache__": true }
}
```

Listing 8.1 — Small on purpose. Add to it when you have a reason, not before.

RULE 08

If the editor and the terminal disagree about your Python, believe the terminal and fix the editor.

Four keystrokes worth learning today

KEYS	DOES
Cmd/Ctrl-Shift-P	Command palette — everything lives here, searchable
Cmd/Ctrl-P	Open any file by typing part of its name
Ctrl-`	Show or hide the built-in terminal
F5	Run the current file under the debugger

The debugger, briefly, because it replaces guessing

Click just left of a line number to set a breakpoint — a red dot appears. Press F5 and choose *Python File*. Execution stops at that line and you can inspect every variable in scope. Ten minutes learning this saves more time than anything else in this step, and it is the habit that separates people who debug from people who add print statements until something changes.

When it goes wrong

SYMPTOM	CAUSE	FIX
Imports underlined in red, but the code runs	The editor is using a different interpreter from the terminal.	Select Interpreter, choose the <code>.venv</code> one, then reload the window.
Runs in the terminal, fails with the Run button	Different interpreter, or a different working folder.	Select the interpreter; open the project folder itself, not a parent.
The built-in terminal does not activate the environment	Setting off, or a shell VS Code cannot hook.	The setting in Listing 8.1, or just activate manually.
Everything is slow, fan spinning	Indexing a huge folder — usually <code>.venv</code> or a model cache.	Open the project folder, not your home directory.
I do not see my <code>.venv</code> in the interpreter list	You opened a parent folder.	File → Open Folder → <code>llm-starter</code> itself.

DO THIS NOW

1. Install VS Code and the Python extension.
2. Open the `llm-starter` folder — the folder itself, not its parent.
3. Select the `.venv` interpreter and confirm it appears in the status bar.
4. Open the built-in terminal with Ctrl-` and check for `(.venv)` in the prompt.
5. Create `.vscode/settings.json` from Listing 8.1.

Verify — the status bar names your `.venv` interpreter, and the built-in terminal shows `(.venv)`.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Opening the wrong folder — a parent, or their whole home directory — after which the interpreter list does not contain their environment and nothing behaves. Watch for anyone whose file tree looks too big. The interpreter picker itself is the other one: people expect the editor to work it out, and it usually does, but “usually” produces the most confusing ten minutes of the afternoon when it does not.

09 CONNECT Git, and a `.gitignore` before anything else

Git comes before the API keys, deliberately. In a moment you will have real secrets on your laptop, and the only reliable way to keep them out of your repository is for the rules that exclude them to exist *before* the files do. Retrofitting is a rewrite of history; doing it now is one file.

What git is, in one paragraph

Git stores snapshots of your whole project, each one pointing at its parent. A file moves through three places: your **working folder** (where you edit), the **staging area** (what will go into the next snapshot), and the **repository** (permanent history). `git add` moves a change from the first to the second; `git commit` moves it from the second to the third. Almost every confusing git moment is not knowing which of those three holds the version you care about — and `git status` answers that every single time.

Configure it once, on this machine

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
$ git config --global init.defaultBranch main
$ git config --global pull.rebase false
$ git config --global --list      # read it back
```

Listing 9.1 — The email here appears in every commit you ever make. Use one you are happy to publish.

Write `.gitignore` now, before the repository exists

Create a file called `.gitignore` in `llm-starter`, containing exactly this:

```
# environment — thousands of machine-specific files
.venv/
__pycache__/
*.pyc

# secrets — never, under any circumstances
.env
.env.*
!.env.example

# model weights and caches — gigabytes, and not source code
models/
*.safetensors
*.gguf
.cache/

# editor and OS noise
.DS_Store
.vscode/*
!.vscode/settings.json

# notebooks' saved output
.ipynb_checkpoints/
```

Listing 9.2 — Read it line by line. The `!` lines are exceptions: “ignore everything like this, except that”.

Four categories, and each earns its place. The environment because it is enormous and machine-specific. Secrets because a key in git history is a key that has been published. Model weights because git is built for text and will punish you for gigabytes of binary. Editor noise because nobody else needs your window layout — except `settings.json`, which records the shared interpreter choice from step 08 and is genuinely useful to a teammate.

Initialise, and check what git can see

```
$ cd ~/projects/llm-starter
$ git init
$ git status
```

Read that `git status` output carefully. You should see `.gitignore` and `.vscode/` — and you should **not** see `.venv`. If `.venv` appears, your ignore file is in the wrong folder or misspelled. Fix that before you commit anything.

```
$ git add .gitignore .vscode/settings.json
$ git commit -m "add gitignore and shared editor settings"
$ git log --oneline
```

RULE 09

The first commit of every project you ever start contains `.gitignore`. Not the second, not “once there is something to ignore”. The first.

The daily commands

COMMAND	WHAT IT DOES
<code>git status</code>	Where everything stands. Run it constantly.
<code>git add file</code> / <code>git add -p</code>	Stage a file / stage changes one piece at a time, reading as you go
<code>git commit -m "message"</code>	Snapshot what is staged
<code>git diff</code> / <code>git diff --staged</code>	What changed but is not staged / what is staged
<code>git log --oneline --graph -20</code>	Recent history with its shape visible
<code>git restore file</code>	Throw away uncommitted changes to a file
<code>git restore --staged file</code>	Unstage without losing the edit
<code>git check-ignore -v file</code>	Why is this file being ignored? Names the rule.
<code>git reflog</code>	Every position you have been in. This is how “lost” work is recovered.

Commit messages that help you later

A subject line under about seventy characters, written as an instruction: “add retry logic”, not “added” or “adding”. If it needs explaining, leave a blank line and write *why* — the diff already says what. You are writing for yourself in six months, wondering why this line exists.

When it goes wrong

SYMPTOM	CAUSE	FIX
Please tell me who you are	No <code>user.email</code> configured.	Listing 9.1.
<code>.venv</code> appears in <code>git status</code>	Ignore file misspelled, or in the wrong folder.	<code>ls -la .gitignore</code> — it must be in the project root. <code>git check-ignore -v .venv</code> explains what git thinks.
I already committed something that should be ignored	Ignore rules do not apply retroactively.	<code>git rm -r --cached .venv</code> , then commit. The files stay on disk.
Every file shows as modified and I changed nothing	Line endings.	The <code>core.autocrlf</code> setting in Listing 9.1, then re-clone or reset.
<code>fatal: not a git repository</code>	You are not inside the project folder.	<code>pwd</code> , then <code>cd ~/projects/llm-starter</code> .
I committed a large model file and cannot push	GitHub rejects very large files.	Remove it from history, add it to <code>.gitignore</code> . Prevention is why step 09 is here and not later.
I think I lost work	Almost never true.	<code>git reflog</code> , find the identifier, <code>git switch -c recovered <id></code> .

DO THIS NOW

1. Set your git identity with the four config commands, then read them back with `--list`.
2. Create `.gitignore` in `llm-starter` with the exact contents of Listing 9.2.
3. Run `git init`, then `git status`, and confirm `.venv` is *not* listed.
4. Make your first commit containing the ignore file and the editor settings.

Verify — `git status` shows a clean tree with no `.venv`, and `git log --oneline` shows one commit.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Two things. People create `.gitignore` in their home folder or inside `.venv` rather than the project root, so it does nothing — have them run `ls -la .gitignore` and read the path aloud. And some editors will silently save it as `gitignore.txt`; if `.venv` still shows, that is the first thing to check. Do not let anyone reach step 12 without a working ignore file: that is the whole reason git is here rather than at the end.

10 CONNECT A GitHub account and an SSH key

You need somewhere to put this project that is not your laptop, and a way of proving to it that you are you. GitHub is the somewhere; an SSH key is the proof. Passwords for git operations no longer work, so this step is not optional.

Create the account

1. Go to **github.com** and choose *Sign up*.
2. Use an email you will keep. Your commits carry it, and changing it later does not rewrite history.
3. Pick a username you would put on a CV. It becomes part of every repository URL you own.
4. Verify the email — nothing works until you do.
5. Turn on two-factor authentication when it offers. GitHub requires it for most accounts anyway, and doing it now is calmer than doing it under pressure.

Generate a key

An SSH key is a pair of files. The **public** half you paste anywhere. The **private** half never leaves your machine — not into a repository, not into a chat message, not onto a shared drive.

```
$ ssh-keygen -t ed25519 -C "you@example.com"
# press Return to accept the default location
# then type a passphrase – yes, use one
$ ls -l ~/.ssh/id_ed25519*      # two files: private, and .pub
$ cat ~/.ssh/id_ed25519.pub    # the PUBLIC half – copy this whole line
```

On macOS, add your key to the agent so the passphrase is remembered by the system keychain:

```
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

Use a passphrase. Your key agent remembers it for the session, so you type it once a day rather than once a push — and a stolen laptop without a passphrase is a stolen GitHub account.

Give GitHub the public half

1. On GitHub, click your avatar → *Settings*.
2. *SSH and GPG keys* → *New SSH key*.
3. Title: name the machine, not the purpose — “work MacBook”, “home laptop”. One key per machine means you can revoke a lost laptop without disturbing anything else.
4. Key type: *Authentication Key*. Paste the whole `ssh-ed25519 ...` line, including the email at the end.

5. Add key.

```
$ ssh -T git@github.com
# first time: type yes to accept the host fingerprint
# success looks like: Hi <username>! You've successfully authenticated...
```

Listing 10.1 — That greeting by name is the whole test. It says: GitHub knows who you are.

Create the empty repository

- 1. On GitHub: *New repository*.
- 2. Name it `llm-starter`.
- 3. Choose **Private** for now. You can make it public later; you cannot un-publish a key.
- 4. Do **not** tick “Add a README”, “.gitignore” or “licence”. You already have a local repository, and those options create a conflicting first commit.
- 5. Create, and leave the page open — you will use its URL in step 18.

RULE 10

The private key never moves. Copy the public half anywhere you like; if a private key has ever been on a shared machine or in a repository, replace it.

When it goes wrong

SYMPTOM	CAUSE	FIX
Permission denied (publickey)	Key not added to GitHub, or not offered by the agent.	<code>ssh-add -l</code> to list loaded keys; <code>ssh -vT git@github.com</code> shows which are tried.
Connection timed out on port 22	A firewall is blocking SSH.	Add a <code>~/.ssh/config</code> entry using <code>Hostname ssh.github.com</code> and <code>Port 443</code> .
WARNING: UNPROTECTED PRIVATE KEY FILE	Permissions too open.	<code>chmod 600 ~/.ssh/id_ed25519</code>
Asked for the passphrase on every command	No agent running.	<code>ssh-add ~/.ssh/id_ed25519</code> . On macOS add <code>UseKeychain yes</code> to <code>~/.ssh/config</code> .
<code>ssh-keygen: command not found</code>	Rare, but happens on minimal installs.	Install OpenSSH client for your platform, or use GitHub Desktop instead.
I pasted the private key by mistake	The file without <code>.pub</code> .	Delete that key on GitHub, generate a new pair, and add only the <code>.pub</code> .

DO THIS NOW

1. Create the GitHub account and verify your email.
2. Generate an ed25519 key with a passphrase.
3. Add the public half to GitHub, named after this machine.
4. Run `ssh -T git@github.com` and confirm it greets you by username.
5. Create the empty private `llm-starter` repository, with no README and no template files.

Verify — `ssh -T git@github.com` prints "Hi <your-username>!"

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Three predictable ones. People paste the private key instead of the public one — say "the file ending `.pub`" twice. People tick "Add a README", then get a rejected push in step 18 with a confusing message about unrelated histories. And the `ssh -T` test prints a message that *looks* like an error ("does not provide shell access") immediately after the success line — read the whole output aloud once so nobody panics.

11

CONNECT

A Hugging Face token

Hugging Face is where open model weights live. You will download a model from it in step 15, and while many models need no token at all, having one takes two minutes and removes a whole category of confusing failure — rate limits on anonymous downloads, and models that require you to accept a licence first.

Create the account and the token

1. Go to **huggingface.co** and sign up. Verify the email.
2. Click your avatar → *Settings* → *Access Tokens*.
3. *Create new token*. Choose the **Read** type — that is all you need to download. Do not create a write token today; you are not uploading anything.
4. Name it for the machine and purpose: `llm-starter-laptop`.
5. Copy it immediately. It is shown **once**. If you lose it, delete it and make another — that is normal and costs nothing.

A read token looks like `hf_` followed by a long string. Treat it exactly like a password: it identifies you, and on a gated model it carries your acceptance of that model's licence.

Where to put it

Do not paste it into any file yet. Step 13 is where secrets get stored properly, and it is the next-but-one step. For now, keep it in your clipboard manager or on paper for five minutes.

When you get there, it goes in `.env` as `HF_TOKEN=hf_...`, and the Hugging Face libraries pick it up from the environment automatically — you will never have to pass it to a function.

Install the client, and log in

```
$ python -m pip install "huggingface_hub[cli]"
$ hf auth login                # paste the token when asked
$ hf auth whoami               # prints your username
```

Listing 11.1 — `hf auth login` stores the token in your home folder. On older versions of the library the command is `huggingface-cli login`; both do the same thing.

Where the models will land — read this before step 15

Model weights are large and they download somewhere invisible. Knowing where, before you pull three gigabytes, is the difference between an orderly disk and a mysterious “disk full” in week two.

```
$ echo ${HF_HOME:~/.cache/huggingface}      # the default cache location
$ du -sh ${HF_HOME:~/.cache/huggingface} 2>/dev/null  # how big it is now
(probably nothing)
$ df -h . | tail -1                          # and how much room you have
```

The model in step 15 is about 3 GB. If you have less than 10 GB free, deal with that now rather than halfway through a download.

Moving the cache, if your disk is small

One environment variable controls all of it. Set it in your shell configuration and every tool obeys:

```
$ echo 'export HF_HOME="/path/to/bigger/disk/hf"' >> ~/.zshrc  # or .bashrc
$ source ~/.zshrc && echo $HF_HOME
```

Cleaning up later

```
$ hf cache scan          # what is cached, and how big
$ hf cache delete        # interactive: choose what to remove
```

Use those rather than deleting folders by hand. The cache uses links between files, and a manual delete can leave it in a state where things re-download without freeing space.

RULE 11

Know where your model cache lives before you fill it. A cache is never data — anything you would be upset to lose does not belong in one.

When it goes wrong

SYMPTOM	CAUSE	FIX
Token shown once and I did not copy it	By design.	Delete it and create another. No harm done.
<code>hf: command not found</code>	Installed outside the active environment, or PATH.	Confirm <code>(.venv)</code> is in your prompt and reinstall. Or run <code>python -m huggingface_hub.commands.huggingface_cli</code> .
<code>401 Unauthorized</code> downloading a model	Not logged in, or the model is gated and you have not accepted its licence.	Visit the model page in a browser, accept the terms, then retry.
<code>403 Forbidden</code> on a gated model	Access request still pending.	Some models need manual approval. Use a non-gated one — step 15 recommends one deliberately.
Download stalls near the end, repeatedly	Network or proxy buffering.	Retry — the client resumes. On a metered connection, download once and share the cache folder.
Disk full mid-download	Weights are larger than the space you had.	<code>hf cache delete</code> , or set <code>HF_HOME</code> to a bigger disk, then retry.
Deleted a model, disk still full	The cache stores blobs and links separately.	<code>hf cache delete</code> , which understands the layout.

DO THIS NOW

1. Create the Hugging Face account and a **read** token. Keep it somewhere safe for five minutes.
2. Install the client into the active environment and run `hf auth login`, then `hf auth whoami`.
3. Run the three cache commands and write down your free disk space. You need at least 10 GB before step 15.

Verify — `hf auth whoami` prints your username, and you can say where model weights will be written on your machine.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

The token being shown once — several people will lose it, and they need to hear that regenerating is routine rather than a failure. The bigger risk is disk: check the room's free space out loud here, because someone always has 4 GB free and will otherwise discover it thirty minutes into step 15. If your cohort is on shared or metered wifi, download the model once onto a USB stick and hand round the cache folder instead of thirty simultaneous 3 GB pulls.

12 CONNECT API keys: OpenAI, Anthropic, Gemini

Three providers, three signup flows, the same shape: create an account, add a payment method or claim free credit, create a key, copy it once. This document's program uses **OpenAI**, so that key is the one you need today — but set up all three while the browser is open, because the differences between them are worth seeing once.

Before you create any key: set the spending limit

Do this first, on every provider, every time. It is the only control that works while you are asleep, and a loop with a bug in it can spend real money faster than you can read a traceback.

OpenAI

1. Go to **platform.openai.com** — this is the developer platform, not the ChatGPT product. A ChatGPT subscription does *not* include API access, and this confuses almost everyone once.
2. Sign up or sign in, then *Settings* → *Billing* and add a small amount of credit. Start with the minimum.
3. *Settings* → *Limits*: set a monthly budget you would be comfortable losing. Five or ten of your local currency units is plenty for this document.
4. *API keys* → *Create new secret key*. Name it `llm-starter`. Scope it to a project if offered.
5. Copy it now. It starts `sk-` and it is shown once.

Cost check, so nothing is a surprise: with a small model, the whole of step 14 — including running it a dozen times while you get it right — costs a fraction of a currency unit. The chat loop you build sends only a few hundred tokens per turn.

Anthropic

1. Go to **console.anthropic.com**. Again: separate from the consumer product.
2. Sign in, then *Plans & Billing* → add credit.
3. Set a spend limit in the same area.
4. *API keys* → *Create key*. Name it, copy it. It starts `sk-ant-`.
5. Note the *Workspaces* feature — it lets you keep coursework separate from anything real, with its own limit. Worth using.

Google Gemini

1. Go to **aistudio.google.com** and sign in with a Google account.
2. *Get API key* → *Create API key*. There is a free tier with real rate limits rather than a bill, which makes this the easiest of the three to try.

3. Copy the key. It usually starts `AIza` .
4. For serious use you would move to Google Cloud and a service-account file instead — out of scope here, but know that the file *is* a secret if you ever download one.

The three side by side

PROVIDER	ENVIRONMENT VARIABLE	KEY LOOKS LIKE	WORTH KNOWING
OpenAI	<code>OPENAI_API_KEY</code>	<code>sk-...</code>	Prepaid credit. Per-project keys and limits. Used by step 14.
Anthropic	<code>ANTHROPIC_API_KEY</code>	<code>sk-ant-...</code>	Workspaces isolate coursework spending.
Gemini	<code>GOOGLE_API_KEY</code>	<code>AIza...</code>	Free tier available — limited by rate, not by bill.
Hugging Face	<code>HF_TOKEN</code>	<code>hf_...</code>	From step 11. Downloads weights; no per-call cost.
Local model	none	—	No key, no cost, no leak. Step 15.

RULE 12

Set the provider's spending limit before you create the key, not after. It is the one protection that does not depend on your code being correct.

Hygiene that costs nothing

- **One key per person per project.** A shared key cannot be revoked without breaking everyone, and cannot tell you who spent the money.
- **Least privilege.** Read-only where reading is all you do — that is why step 11 asked for a read token.
- **Never print a key.** Not to check it worked. Log the last four characters if you must.
- **Rotate on any doubt.** Deleting a key and making another takes thirty seconds.

When it goes wrong

SYMPTOM	CAUSE	FIX
I have ChatGPT Plus but the API says I have no credit	They are separate products with separate billing.	Add credit on platform.openai.com. This is the single most common confusion here.
429 insufficient_quota on the very first call	No credit, or a spend limit already reached.	Check the billing page. A brand-new account sometimes needs a few minutes after payment.
Key shown once and I lost it	By design.	Delete it, create another. Never email one to yourself.
401 invalid_api_key	Wrong variable name, whitespace in the value, or a revoked key.	Step 13 shows how to check what is loaded without printing it.
Country or payment method not supported	Provider availability varies.	Use Gemini's free tier, or the local model in step 15. Neither needs a card.
I do not want to enter a card at all	Reasonable.	Gemini's free tier for the API step, and step 15 needs no key whatsoever. You can complete this document without paying anything.

DO THIS NOW

1. Set a spending limit on OpenAI, then create one key named `llm-starter`. Keep it in your clipboard.
2. If you want them, create the Anthropic and Gemini keys too — the same pattern each time.
3. Do **not** paste any key into a file yet. Step 13 is next, and it is one page.

Verify — you have at least one API key in your clipboard, and a spending limit set on the account it belongs to.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

The ChatGPT-versus-API distinction, without exception, every cohort. Address it before anyone opens a browser. Second: payment. Some people cannot or will not add a card, and they must not feel stuck — point them at Gemini's free tier and step 15's local model, and say plainly that the document is completable with no spending at all. Third: someone will paste a key into the class chat. Have the leak procedure in step 13 ready to run as a live demonstration; it turns an embarrassment into the most memorable ten minutes of the day.

13 CONNECT `.env`, and what to do when a key leaks

You now have keys in your clipboard and a repository with an ignore file. This step connects them safely, and then tells you exactly what to do on the day it goes wrong — because in any group of people learning this, it goes wrong for someone, and the difference between a non-event and a real problem is entirely about the first ten minutes.

The rule

A secret never appears in source code, in a notebook cell, in a commit, in a screenshot, in a chat message, or inside a container image. It lives in an environment variable, loaded from a file that git ignores.

Create the two files

`.env.example` is committed and lists the *names* with no values, so a newcomer knows what the project needs without you sending them anything. `.env` holds the real values and is never committed.

```
$ cd ~/projects/llm-starter

$ cat > .env.example <<'EOF'
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
GOOGLE_API_KEY=
HF_TOKEN=
EOF

$ cp .env.example .env
$ # now open .env in your editor and paste the real values after the = signs
$ chmod 600 .env                # only you can read it
$ git status                    # .env must NOT appear. .env.example should.
```

Listing 13.1 — That `git status` is the most important check on this page. If `.env` is listed, stop and fix step 09 before continuing.

Format matters slightly: no spaces around the `=`, no quotes needed, one per line.

`OPENAI_API_KEY=sk-abc123`, not `OPENAI_API_KEY = "sk-abc123"`.

Load it in Python

```
import os
from dotenv import load_dotenv

load_dotenv()                                # reads .env from the current folder

key = os.environ.get("OPENAI_API_KEY")
if not key:
    raise SystemExit(
        "OPENAI_API_KEY is not set. Copy .env.example to .env and fill it in."
    )
print(f"key loaded, ending ...{key[-4:]}")    # last four characters only. Never
the key.
```

Listing 13.2 — Save this as `check_keys.py` and run it. That explicit error message is worth more than a stack trace six frames deep, and you will thank yourself for it in step 14.

```
$ python check_keys.py
key loaded, ending ...a1b2
```

RULE 13

Never print a key to confirm it works. Print its last four characters, or print nothing. Terminal output ends up in logs, screenshots and screen shares.

Commit the safe half

```
$ git add .env.example check_keys.py
$ git commit -m "add env template and key check"
$ git log --oneline
$ git show --stat HEAD                # confirm .env is not in there
```

You leaked a key. Do this, in this order.

This will happen to someone. It is recoverable, and the order matters more than the speed.

1. **Revoke it at the provider.** Now, before anything else. A revoked key cannot be misused, and no amount of git surgery achieves that. Deleting the commit does not un-publish the key.
2. **Create a new one** and put it in `.env`.
3. **Check the usage dashboard** for calls you did not make.
4. **Then** clean the history — `git filter-repo`, or GitHub's own guidance — and tell anyone with a clone to re-clone. This is tidying, not remediation.
5. **Fix the cause.** The missing ignore rule, the printed variable, the copied file.

6. Tell someone. Your instructor, or your security team. Quietly hoping is how a small incident becomes a large one.

```
$ # check your own history for anything that looks like a key
$ git log -p | grep -nE "sk-[A-Za-z0-9]|hf_[A-Za-z0-9]|AIza" || echo "clean"
```

Listing 13.3 — Run this now, while the answer is “clean”, so you know the command when you need it.

Where secrets live on other machines

SITUATION	WHERE THE KEY GOES
Your laptop	<code>.env</code> , git-ignored, permissions locked down
A notebook (Colab)	The Secrets panel, never a cell
A Docker container	Passed in at run time — step 17. Never copied into the image.
Automated builds	The platform's own secret store
A teammate	Their own key. Never yours.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>.env</code> shows up in <code>git status</code>	Ignore rule missing or misspelled.	Fix <code>.gitignore</code> (step 09) before you commit anything. <code>git check-ignore -v .env</code> explains what git sees.
Key loads in the terminal but not in VS Code	Different working folder, so <code>load_dotenv()</code> looked elsewhere.	Open the project folder itself; or use <code>load_dotenv(override=True)</code> with an explicit path.
<code>401</code> even though the key is in <code>.env</code>	Whitespace, quotes, or a stray blank line in the file.	No spaces around <code>=</code> , no quotes. Re-copy the key.
<code>ModuleNotFoundError: dotenv</code>	Package installed outside the active environment.	Check for <code>(.venv)</code> , then <code>python -m pip install python-dotenv</code> .
I already pushed <code>.env</code>	It is public now, whatever the repository visibility says.	Revoke the keys first. Then clean history. In that order.
A key appeared in a screenshot I shared	It was on screen.	Revoke it. Screenshots are archived and indexed.

DO THIS NOW

1. Create `.env.example` and `.env`, paste your real keys into the second one, and lock its permissions.
2. Run `git status` and confirm `.env` does not appear.
3. Write `check_keys.py` from Listing 13.2 and run it. It should print four characters and nothing more.
4. Commit `.env.example` and `check_keys.py`. Confirm with `git show --stat` that `.env` is not in the commit.
5. Run the grep in Listing 13.3 and see it say "clean".

Verify — `python check_keys.py` prints a four-character suffix, and no secret exists anywhere in git.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Formatting: quotes and spaces around the `=`, or a trailing space in a pasted key, which produces a `401` that looks like a provider problem. Show a broken `.env` and a working one side by side. Then rehearse the leak procedure as a group — create a throwaway key, commit it deliberately, revoke it, and clean up. Ten minutes, and the room will never panic about it again. Say the ordering out loud twice: **revoke first, tidy second**. People instinctively reach for git.

14 BUILD chat_openai.py — a model over the internet

Everything so far has been preparation. This is the payoff: forty lines that send what you type to a language model and print what comes back, one word at a time, until you tell it to stop.

What you are building

A loop. Read a line from the keyboard, send it to the model along with everything said so far, stream the reply to the screen, repeat. Two commands: `/quit` to leave and `/reset` to forget the conversation and start fresh. Nothing else — every feature you add is another thing to explain when it breaks.

Install the one package you still need

```
$ cd ~/projects/llm-starter
$ # confirm (.venv) is in your prompt before this next line
$ python -m pip install openai python-dotenv
```

You installed both in step 06, so this should report that they are already satisfied. That is a useful confirmation that you are in the right environment.

The program

Create `chat_openai.py` in the project folder and type this in. *Type* it, at least the first time — the twenty minutes of typing teaches you more than the four seconds of pasting.

```
"""A minimal streaming chat loop against the OpenAI API."""
import os
import sys

from dotenv import load_dotenv
from openai import OpenAI

MODEL = "gpt-4o-mini"          # small, fast, inexpensive — right for learning
SYSTEM = "You are a concise, helpful assistant. Keep answers short."

def main() -> None:
    load_dotenv()
    key = os.environ.get("OPENAI_API_KEY")
    if not key:
        sys.exit("OPENAI_API_KEY is not set. Copy .env.example to .env and fill it in.")
```

```

client = OpenAI(api_key=key, timeout=60)
history = [{"role": "system", "content": SYSTEM}]

print(f"Talking to {MODEL}. /reset to start over, /quit to leave.\n")

while True:
    try:
        user = input("you > ").strip()
    except (EOFError, KeyboardInterrupt):
        print("\nbye")
        return

    if not user:
        continue
    if user == "/quit":
        print("bye")
        return
    if user == "/reset":
        history = [{"role": "system", "content": SYSTEM}]
        print("(conversation cleared)\n")
        continue

    history.append({"role": "user", "content": user})

    print("bot > ", end="", flush=True)
    reply = ""
    stream = client.chat.completions.create(
        model=MODEL,
        messages=history,
        max_tokens=400,          # a ceiling on the expensive half of the bill
        stream=True,
    )
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
            reply += piece
    print("\n")

    history.append({"role": "assistant", "content": reply})

if __name__ == "__main__":
    main()

```

Listing 14.1 — `chat_openai.py`. Forty lines, and nothing in it is decoration.

Run it

```
$ python chat_openai.py
Talking to gpt-4o-mini. /reset to start over, /quit to leave.

you > explain a virtual environment in two sentences
bot > A virtual environment is a folder holding its own Python ...

you > /quit
bye
```

Read the parts that matter

The `history` list is the whole of “memory”. The model has none of its own — every call sends the entire conversation again. That is why long chats cost more per turn than short ones, and why `/reset` is a genuinely useful command rather than a nicety.

The three roles. `system` sets standing instructions, `user` is you, `assistant` is the model. Appending the model's own reply to the history is what makes the next turn coherent.

`stream=True` changes the shape of the response from one finished answer into a series of small pieces. The words appear as they are generated, which makes a two-second wait feel instant. The `flush=True` matters — without it Python holds the text in a buffer and you get the whole line at once anyway.

`max_tokens=400` is a cost ceiling, not a suggestion. Output is billed at a higher rate than input, and a model asked an open-ended question will happily write an essay.

`timeout=60` means a stalled network fails in a minute rather than hanging forever. Never leave a network call without one.

Understanding what a turn costs

Providers bill per *token* — roughly three-quarters of a word in English — and charge more for output than input. Two habits keep the number small while you are learning: keep the system prompt short, because it is resent on every single turn; and use `/reset` when you change subject rather than dragging an hour of conversation into every request.

To see the numbers rather than guess them, remove `stream=True` temporarily and print `response.usage` — it reports input and output tokens for that call. The provider's dashboard shows the running total, and looking at it once after this exercise is the cheapest cost lesson available.

When the provider says no

Three responses you will meet, and the right reaction to each. This is the difference between a script and something you can rely on.

STATUS	MEANING	WHAT TO DO
401	Bad or missing key	Never retry. The same request will fail identically. Step 13.
429	Too many requests, or out of credit	Wait and retry, doubling the wait each time. If it is credit, no amount of waiting helps.
400	Malformed request — often too many tokens	Never retry. Shorten the conversation, or <code>/reset</code> .
500 , 503	The provider's own failure	Retry a few times with a growing wait. Then stop.
Timeout	Slow generation or a network stall	Retry once. This is why <code>timeout=60</code> exists.

The minimal loop above does not retry, on purpose — you should see a real 429 once and understand it. When you do want retries, the pattern is: catch the error, wait one second, then two, then four, adding a small random amount each time so that parallel workers do not all retry in lockstep.

RULE 14

Retry on 429 and 5xx. Never retry on 401, 400 or 404 — you will pay for the privilege of the same failure.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>ModuleNotFoundError: openai</code>	Environment not active, or installed elsewhere.	Check for <code>(.venv)</code> ; <code>python -m pip show openai</code> and read its Location.
<code>OPENAI_API_KEY is not set</code>	No <code>.env</code> , or you are running from another folder.	<code>pwd</code> ; confirm <code>.env</code> is beside the script. Step 13.
<code>401 invalid_api_key</code>	Whitespace or quotes in <code>.env</code> , or a revoked key.	Re-copy the key with no quotes and no trailing space.
<code>429 insufficient_quota</code>	No credit on the account.	Add credit, switch to Gemini's free tier, or skip to step 15 which needs no key.
<code>404 model_not_found</code>	That model name is not available to your account.	Try <code>gpt-4o-mini</code> ; check the models list in your dashboard.
Reply appears all at once instead of streaming	Missing <code>flush=True</code> .	Add it to both <code>print</code> calls.
<code>context_length_exceeded</code> after a long chat	The history has outgrown the model's window.	<code>/reset</code> . Or keep only the last few turns.
Ctrl-C leaves a mess on screen	Interrupt mid-stream.	Harmless. The <code>KeyboardInterrupt</code> handler catches it at the prompt.
It works but replies are strange or too long	The system prompt.	Edit <code>SYSTEM</code> . The cheapest experiment in this document — try five versions.

DO THIS NOW

1. Type `chat_openai.py` out in full and run it. Ask it something.
2. Ask a follow-up that only makes sense given your first question — “explain that more simply” — and confirm it has the context.
3. Use `/reset` , ask the same follow-up, and watch it lose the thread. That contrast is the lesson about history.
4. Change `SYSTEM` to something with personality, restart, and see how much behaviour shifts for one line of text.
5. Commit it: `git add chat_openai.py && git commit -m "add streaming chat against OpenAI"`

Verify — you can hold a two-turn conversation, and `/reset` demonstrably clears the memory.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Typing errors in a forty-line file, mostly indentation — Python is unforgiving and the messages are terse. Have them run it after each block rather than at the end. The conceptual moment worth pausing on is the history list: many people assume the model remembers, and the `/reset` contrast is what makes it click. If someone has no credit, pair them with someone who does for this step and make sure they reach step 15, which needs no key at all.

15 BUILD `chat_local.py` — a model on your laptop

Same program, no internet, no key, no cost. The model runs on your own hardware, which changes the trade-offs completely: nothing you type leaves the machine, and nothing you do is metered — but the model is much smaller, so the answers are noticeably weaker, and the first run downloads about three gigabytes.

Check you have room first

```
$ df -h . | tail -1          # need 10 GB free, comfortably
$ echo ${HF_HOME:~/.cache/huggingface} # where it will land
```

If that is tight, revisit the `HF_HOME` section of step 11 before you start the download rather than after it fails.

Install the libraries

```
$ python -m pip install torch transformers accelerate
```

On Apple Silicon this is straightforward — the default `torch` package includes Apple GPU support, so there is no special index or CUDA version to choose. The download is a few hundred megabytes.

The model

We use **Qwen2.5-1.5B-Instruct**: about 1.5 billion parameters, roughly 3 GB on disk, no licence gate, and unusually capable for its size. “Instruct” matters — it means the model has been trained to follow instructions and hold a conversation. A base model of the same size would just continue your text rather than answer you.

For scale: this is somewhere around a thousandth the size of the largest models you have used through a website. Expect it to be useful for summarising, rewriting and simple questions, and poor at reasoning, arithmetic and anything needing world knowledge. That contrast is the point of running it.

The program

```
"""The same chat loop, against a model running on this machine."""
from threading import Thread

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, TextIteratorStreamer
```

```
MODEL_ID = "Qwen/Qwen2.5-1.5B-Instruct"
SYSTEM = "You are a concise, helpful assistant. Keep answers short."

def pick_device() -> str:
    if torch.backends.mps.is_available():
        return "mps"          # Apple Silicon GPU
    return "cpu"

def main() -> None:
    device = pick_device()
    print(f"loading {MODEL_ID} on {device} – first run downloads about 3 GB ...")

    tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_ID,
        torch_dtype=torch.float16 if device != "cpu" else torch.float32,
    ).to(device)

    history = [{"role": "system", "content": SYSTEM}]
    print("ready. /reset to start over, /quit to leave.\n")

    while True:
        try:
            user = input("you > ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nbye")
            return

        if not user:
            continue
        if user == "/quit":
            print("bye")
            return
        if user == "/reset":
            history = [{"role": "system", "content": SYSTEM}]
            print("(conversation cleared)\n")
            continue

        history.append({"role": "user", "content": user})

        prompt = tokenizer.apply_chat_template(
            history, tokenize=False, add_generation_prompt=True
        )
        inputs = tokenizer(prompt, return_tensors="pt").to(device)
```

```

streamer = TextIteratorStreamer(
    tokenizer, skip_prompt=True, skip_special_tokens=True
)
thread = Thread(
    target=model.generate,
    kwargs=dict(**inputs, streamer=streamer, max_new_tokens=300),
)
thread.start()

print("bot > ", end="", flush=True)
reply = ""
for piece in streamer:
    print(piece, end="", flush=True)
    reply += piece
thread.join()
print("\n")

history.append({"role": "assistant", "content": reply})

if __name__ == "__main__":
    main()

```

Listing 15.1 — `chat_local.py`. Read it beside Listing 14.1: the loop, the history and the commands are identical. Only the middle changed.

Run it

```

$ python chat_local.py
loading Qwen/Qwen2.5-1.5B-Instruct on mps — first run downloads about 3 GB ...
ready. /reset to start over, /quit to leave.

you > summarise what a virtualenvironment is
bot > A virtual environment is an isolated folder ...

```

The first run is slow — it is downloading. Every run after that loads from the cache in seconds.

What changed, and why

The chat template. The API in step 14 took a list of messages and handled the formatting itself. Here you are talking to raw weights, and each model family expects its conversation laid out in its own particular way. `apply_chat_template` reads that format from the model's own files, which is why it works across models without you memorising anything.

Tokenising, explicitly. Models do not read text; they read numbers. `tokenizer(...)` converts your string into those numbers and the streamer converts them back. In step 14 this happened on someone else's server.

The thread. Generation blocks until it is finished, so streaming needs it running elsewhere while the main program reads pieces as they arrive. This is the only genuinely fiddly part of the file.

The device. On Apple Silicon, `mps` is Apple's GPU interface — it uses the graphics cores built into your chip and is several times faster than the CPU path. It is part of macOS; there is nothing to install. Occasionally an operation is not yet implemented for `mps`; if you hit that, set `PYTORCH_ENABLE_MPS_FALLBACK=1` in your shell and it will quietly use the CPU for that one operation.

float16. Each parameter stored in two bytes rather than four — half the memory, and on this hardware, faster. It is why 1.5 billion parameters fit in roughly 3 GB.

The honest comparison

	API (STEP 14)	LOCAL (STEP 15)
Cost per message	Small but real, and it accumulates	Nothing, ever
Privacy	Your text goes to a company	Nothing leaves the laptop
Quality	Much better	Useful for simple tasks; weak beyond them
Speed	Fast, network-dependent	Depends entirely on your hardware
Setup	A key	3 GB and some patience
Works offline	No	Yes — try it with wifi off
Good for	Anything needing quality	Bulk work, private data, iterating without a bill

RULE 15

Prototype against a local model when you will run something thousands of times; reach for the API when quality is what matters. Knowing which problem you have is the skill.

When it goes wrong

SYMPTOM	CAUSE	FIX
Download stops partway	Network interruption.	Rerun the script. It resumes.
No space left on device	Cache larger than your free disk.	<code>hf cache delete</code> , or set <code>HF_HOME</code> to a bigger disk. Step 11.
Killed with no other message	The system ran out of memory.	Close browsers. Try <code>torch_dtype=torch.float32</code> on CPU, or a smaller model.
Extremely slow — a word every few seconds	Running on CPU with little memory.	Expected on modest hardware. Lower <code>max_new_tokens</code> to 150 and ask shorter questions.
<code>ModuleNotFoundError: torch</code>	Install went elsewhere.	Check for <code>(.venv)</code> and reinstall.
Replies are repetitive or nonsensical	A 1.5B model at the edge of its ability.	Not a bug. Simplify the question — and note the contrast with step 14.
401 or 403 from Hugging Face	A gated model, or not logged in.	This model is not gated. Check the ID for typos; <code>hf auth whoami</code> .
Second run still downloads	<code>HF_HOME</code> changed between runs.	Set it once in your shell configuration, not per-command.
<code>NotImplementedError</code> mentioning <code>mps</code>	That operation is not implemented for Apple GPU yet.	Run with <code>PYTORCH_ENABLE_MPS_FALLBACK=1</code> set.
Fans loud, machine hot	Sustained GPU work.	Normal. Lower <code>max_new_tokens</code> if it bothers you.

DO THIS NOW

1. Confirm your free disk space, then write `chat_local.py` and run it. Expect a wait on the first run.
2. Ask it the same question you asked in step 14. Note the difference in quality honestly — that comparison is the exercise.
3. Turn off your wifi and run it again. It still works. Sit with that for a second.
4. Ask it something requiring arithmetic or recent facts, and watch it fail. Now you know what a small model is for.
5. Commit it: `git add chat_local.py && git commit -m "add local model chat with Qwen2.5-1.5B"`

Verify — both scripts run, and you can state in one sentence when you would choose each.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Bandwidth, first and foremost. Thirty simultaneous 3 GB downloads will destroy a classroom connection — download once beforehand and distribute the cache folder, or stagger the room in halves. Second, the wait: people assume a silent five-minute download has hung. Tell them what normal looks like. Third, disappointment — the replies are much worse than the API and some conclude they have done it wrong. Frame it in advance as the point of the exercise: this is what a thousandth of the size feels like.

BUILD

16 Freezing requirements.txt

Your project works on your laptop. Right now nobody else can reproduce it — including you, on a new machine, in six months. This step fixes that, and it is the shortest genuinely important step in the document.

The two files, and why both

`requirements.in` is what you *asked for*: a handful of lines, written by hand, recording intent.

`requirements.txt` is what you *got*: every package including the ones that arrived as dependencies, at exact versions. Reviewers read the first; machines read the second.

```
$ cd ~/projects/llm-starter
$ cat > requirements.in <<'EOF'
openai
python-dotenv
torch
transformers
accelerate
huggingface_hub
EOF

$ python -m pip freeze > requirements.txt
$ wc -l requirements.in requirements.txt
```

Listing 16.1 — Six lines in one file, likely forty or more in the other. That gap is exactly the information you would otherwise lose.

Read what freeze produced

```
$ head -12 requirements.txt
accelerate==1.0.1
annotated-types==0.7.0
anyio==4.6.2
certifi==2024.8.30
...
```

Every line is pinned with `==`. That is the whole value: install from this file on another machine and you get the same versions, not merely the same names. Without it, someone setting up your project next month gets whatever is newest, and “works on my machine” becomes a real conversation.

Prove it works

This is the part people skip, and it is the only part that actually verifies anything.

```
$ deactivate
$ rm -rf .venv                # yes, really. It is disposable.
$ python3.11 -m venv .venv
$ source .venv/bin/activate
$ python -m pip install -r requirements.txt
$ python chat_openai.py      # still works?
```

Listing 16.2 — Delete and rebuild. If this succeeds, your project is reproducible. If it does not, you have found the gap now rather than in front of a colleague.

RULE 16

If you cannot delete your environment and rebuild it with one command, you do not have an environment — you have a pet.

Keeping it honest

- **Re-freeze whenever you install something.** Install, test, freeze, commit — in that order, as one habit.
- **Never edit `requirements.txt` by hand.** Change `requirements.in`, install, re-freeze.
- **Commit both files.** They answer different questions.
- **Do not freeze from a polluted environment.** If you have installed things outside the project, `freeze` will capture them. Rebuild first if in doubt.

What reproducibility does not cover

Pinned packages are the largest part of the problem, not all of it. Three more things belong in the repository if a rebuild is to mean anything: the Python version, any non-Python system dependency, and a note about the data. Your `README.md` is the right place for the last two.

```
$ echo "3.11" > .python-version
$ cat > README.md <<'EOF'
# llm-starter
```

Two small chat programs: one against the OpenAI API, one against a local Hugging Face model.

Setup

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env      # then add your keys
```

Run

```
python chat_openai.py      # needs OPENAI_API_KEY
python chat_local.py       # no key; downloads ~3 GB on first run
```

EOF

```
$ git add requirements.in requirements.txt .python-version README.md
$ git commit -m "pin dependencies and document setup"
```

Read that README as if you had never seen the project. Two commands to set up, two to run. That is the standard to hold yourself to on every project from here on.

When it goes wrong

SYMPTOM	CAUSE	FIX
<code>freeze</code> output includes packages I never installed	Environment not active when you installed, so it captured global packages.	Rebuild <code>.venv</code> from scratch, install from <code>requirements.in</code> , freeze again.
<code>requirements.txt</code> has a line pointing at a local path	An editable or local install.	Fine for you, useless to others. Fix <code>requirements.in</code> and re-freeze.
Rebuild fails on another machine	A pinned version has no build for that platform.	Common with <code>torch</code> across operating systems. Loosen just that pin, or document a per-platform install line.
<code>pip install -r</code> is extremely slow	Large packages — <code>torch</code> is hundreds of megabytes.	Normal. <code>uv pip install -r</code> is dramatically faster if you installed <code>uv</code> in step 07.
I deleted <code>.venv</code> and now nothing works	You have not rebuilt it yet.	Listing 16.2, the four commands. This is the drill working as intended.
Two people get different versions from the same file	Someone installed without <code>-r</code> , or edited the file by hand.	Rebuild both environments from the committed file.

DO THIS NOW

1. Write `requirements.in` with your six direct dependencies.
2. Freeze the environment into `requirements.txt` and read the first dozen lines.
3. Delete `.venv` completely, rebuild it from the file, and run `chat_openai.py` to prove it.
4. Write `.python-version` and `README.md`, then commit all four files.

Verify — a full rebuild from an empty `.venv` ends with a working chat program, using only committed files.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

Nobody wants to delete a working environment, so almost everyone skips Listing 16.2 — and then the reproducibility claim is untested. Make it mandatory and do it together; the relief when it rebuilds is what installs the habit. Watch for anyone who installed packages before activating in step 06: their `freeze` will contain surprises, and rebuilding is the fastest cure. One nuance to flag: `torch` pins do not always transfer between platforms, so if your cohort is mixed, agree now whether the committed file is per-platform or whether torch gets a looser pin.

17 SHIP Docker, one image, both scripts

Your project works on your laptop because your laptop has exactly the right Python, the right packages and the right folder layout. A container packages all of that together so it runs the same way on any machine with Docker — which is how software gets handed to a server, a colleague, or yourself in two years.

Three words, precisely

WORD	MEANING
Image	A read-only filesystem plus a default command. Built once, shared, versioned by a tag.
Container	A running instance of an image. It gets its own writable layer, thrown away when it stops.
Volume	Storage that outlives containers. Where anything you want to keep belongs.

A container is not a virtual machine. It shares your machine's kernel, starts in a fraction of a second, and — this is the part that confuses everyone once — **cannot see your files unless you deliberately hand them over**. The container's `/app` is not your `llm-starter`.

Install Docker

Download **Docker Desktop for Mac — Apple Silicon** from docker.com and install it. It asks for an administrator password once, during installation.

```
$ docker --version
$ docker run --rm hello-world # the whole install test
```

No admin. Docker Desktop cannot be installed without a password, and there is no user-level substitute worth teaching here. Read this step for the concepts, skip the commands, and carry on to step 18 — nothing later depends on it.

The `.dockerignore` — write this first

Before the Dockerfile. Without it, `docker build` hands your *entire* project to the build engine — including `.venv`, your git history, and `.env`. That is both slow and dangerous.

```
$ cd ~/projects/llm-starter
$ cat > .dockerignore <<'EOF'
.venv/
.git/
.env
.env.*
__pycache__/
*.pyc
models/
.cache/
.vscode/
README.md
EOF
```

Listing 17.1 — Note `.env` on that list. A secret copied into an image is a secret published to everyone who ever pulls it.

The Dockerfile

One image, both scripts, chosen at run time. Create a file called `Dockerfile` — no extension:

```
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    HF_HOME=/cache/huggingface

WORKDIR /app

# dependencies first — this layer is cached until requirements.txt changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# then the code, which changes on every commit
COPY chat_openai.py chat_local.py ./

# the switch: default to the API script, override at run time
ENTRYPOINT ["python"]
CMD ["chat_openai.py"]
```

Listing 17.2 — Layer order is the whole performance story. Dependencies before code means editing a script rebuilds in two seconds rather than two minutes.

`PYTHONUNBUFFERED=1` matters more than it looks: without it Python holds output in a buffer and your streaming chat appears frozen inside a container. `ENTRYPOINT` plus `CMD` is the switch — the entrypoint is always `python`, and the command after it is replaceable.

Build it

```
$ docker build -t llm-starter:0.1 .  
$ docker images | head -3          # how big did it get?
```

The first build takes a few minutes — it is downloading Python and installing `torch`. Now change one line of `chat_openai.py` and build again: seconds. That is the layer cache, and it is why the `COPY` order is deliberate.

Run it — the API script

```
$ docker run --rm -it --env-file .env llm-starter:0.1  
# --rm          delete the container when it exits  
# -it           interactive, with a terminal — required for a chat loop  
# --env-file    hand the keys in at run time. They are NOT in the image.
```

Run it — the local model

```
$ docker volume create hfcache  
$ docker run --rm -it \  
  -v hfcache:/cache/huggingface \  
  llm-starter:0.1 chat_local.py
```

Listing 17.3 — The trailing `chat_local.py` replaces the `CMD`. That is the switch in action: one image, two programs.

The named volume is what stops the model re-downloading three gigabytes on every single run. Without it the weights land in the container's writable layer and vanish when it exits. This is the difference between a container that is usable and one that is merely correct.

Apple Silicon has **no GPU passthrough** to Linux containers. Inside a container, `chat_local.py` runs on CPU and is noticeably slower than it is natively, where it uses `mps`. That is a real limitation, not a misconfiguration — run the local model natively and use containers for the API script and for shipping.

The commands worth knowing

COMMAND	WHAT IT DOES
<code>docker build -t name:tag .</code>	Build from the Dockerfile here
<code>docker run --rm -it name:tag</code>	Run interactively, clean up on exit
<code>docker run --rm -it name:tag bash</code>	Get a shell inside and look around — the best debugging tool here
<code>docker ps -a</code>	What is running, and what is lying about
<code>docker logs <id></code>	A container's output
<code>docker images</code>	What you have built, and how large
<code>docker system df</code>	Where the gigabytes went
<code>docker system prune</code>	Reclaim them. Read the warning first.

RULE 17

Code goes in the image, secrets go in at run time, data goes in a volume. Mixing those three up is how images become both enormous and unsafe.

Do you actually need this?

Honestly: not for coursework on your own laptop — `requirements.txt` already does that job with less ceremony. You need it when you deploy, when a team must share one environment exactly, or when a project needs system libraries that fight your operating system. Learn it now, while the stakes are a chat script, so it is familiar when the stakes are a deadline.

Commit it

```
$ git add Dockerfile .dockerignore
$ git commit -m "add container image running either chat script"
```

When it goes wrong

SYMPTOM	CAUSE	FIX
Cannot connect to the Docker daemon	Docker is not running.	Open Docker Desktop from Applications and wait for the whale icon in the menu bar to settle.
Build is slow every single time	Layer order, or no <code>.dockerignore</code> .	Dependencies before code; add the ignore file from Listing 17.1.
Container exits immediately	You forgot <code>-it</code> , so <code>input()</code> got no terminal.	Add <code>-it</code> . A chat loop needs it.
<code>OPENAI_API_KEY</code> is not set inside the container	The image cannot see your <code>.env</code> — correctly.	Pass it: <code>--env-file .env</code> .
The model re-downloads on every run	No volume, so the cache lives in the throwaway layer.	The named volume in Listing 17.3.
Streaming output appears all at once	Python buffering.	<code>PYTHONUNBUFFERED=1</code> — it is in Listing 17.2 for exactly this reason.
Image is enormous, several gigabytes	Mostly <code>torch</code> . Some of it is avoidable.	Use the CPU-only torch build inside the image; check with <code>docker images</code> .
<code>.env</code> ended up inside the image	Missing <code>.dockerignore</code> .	Revoke the keys first (step 13), fix the ignore file, rebuild, delete the old image.
<code>exec format error</code>	The image was built for Intel.	Rebuild locally, or add <code>--platform linux/arm64</code> . Avoid <code>linux/amd64</code> unless you must — it emulates, and it is slow.
Container is far slower than running natively	No GPU passthrough on Apple Silicon.	Expected. Run <code>chat_local.py</code> outside the container.

DO THIS NOW

1. Install Docker and confirm `docker run --rm hello-world` works.
2. Write `.dockerignore` first, then the `Dockerfile`.
3. Build the image and note its size.
4. Run the API script in a container with `--env-file .env`, and hold a short conversation.
5. Create the named volume and run `chat_local.py` in a container. Run it twice — the second time should not re-download.
6. Get a shell inside with `bash` and confirm `.env` is **not** in `/app`.
7. Commit both files.

Verify — one image runs both scripts, and no secret exists anywhere inside it.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

The missing `-it`, guaranteed: the container starts, `input()` gets no terminal, and it exits instantly with no error — which looks like a broken build. Mention it before they run anything. Second, the mental model of isolation: people expect the container to see their files and their environment variables. The `bash` exercise fixes this in thirty seconds and is worth doing as a group. Third, if anyone lacks admin rights, have them read the step and skip the commands — nothing later depends on it.

18 SHIP Pushing to GitHub

Last step. Your work exists in one place, on one laptop, protected by nothing. Ten minutes fixes that permanently.

Look at what you are about to publish

Do this before connecting anything. Once pushed, assume it is public forever, whatever the repository's visibility setting says.

```
$ cd ~/projects/llm-starter
$ git status                # anything uncommitted?
$ git ls-files              # everything git tracks. Read every line.
```

That list should contain roughly ten files: the two chat scripts, `check_keys.py`, `requirements.in` and `requirements.txt`, `.gitignore`, `.dockerignore`, `Dockerfile`, `.env.example`, `.python-version`, `README.md`, and `.vscode/settings.json`.

It must **not** contain `.env`, `.venv`, or anything ending `.safetensors`. If it does, stop. Fix it before you push, not after.

```
$ git log -p | grep -nE "sk-[A-Za-z0-9]|hf_[A-Za-z0-9]|AIza[A-Za-z0-9]" || echo
"clean"
```

Listing 18.1 — The last check. “clean” means no key has ever been committed, at any point in your history.

Connect the remote and push

```
$ git remote add origin git@github.com:<your-username>/llm-starter.git
$ git remote -v                # read it back — typos here are common
$ git branch -M main           # make sure the branch is called main
$ git push -u origin main
```

Listing 18.2 — The `-u` sets the upstream, which is why every future push is just `git push`.

Refresh the repository page in your browser. Your files are there, your README renders on the front page, and your commit history is readable. That is the deliverable.

The everyday loop from here

```
$ # edit something
$ git status
$ git add chat_openai.py
$ git commit -m "shorten the system prompt"
$ git push
```

Four commands, in that order, for the rest of your career. Commit small and often; a commit is a save point, not a publication.

Working on a branch, briefly

For anything you are unsure about, work on a branch so `main` always runs:

```
$ git switch -c feature/add-retries
$ # ... edit, commit ...
$ git push -u origin feature/add-retries
$ # then open a pull request on GitHub, and merge it when you are happy
$ git switch main && git pull
```

RULE 18

Read `git ls-files` before your first push to any new repository. It takes five seconds, and it is the last moment at which a mistake is still private.

Finish the README

One more paragraph makes this project genuinely useful to someone else — including you, later. Add a short note on what it does, what it costs, and the two things that surprise people:

```
## Notes

- chat_openai.py needs credit on an OpenAI account. A short
  conversation costs a fraction of a currency unit.
- chat_local.py needs no key and no network after the first run,
  but downloads about 3 GB of model weights the first time.
- Both keep conversation history in memory only. /reset clears it,
  /quit exits.
```

```
$ git add README.md && git commit -m "document cost and first-run behaviour" && git
push
```

When it goes wrong

SYMPTOM	CAUSE	FIX
Permission denied (publickey)	SSH key not registered, or not loaded.	Step 10. <code>ssh -T git@github.com</code> should greet you by name.
remote origin already exists	You ran <code>remote add</code> twice.	<code>git remote set-url origin <correct-url></code>
failed to push some refs / fetch first	The GitHub repository has commits yours does not — usually a README added at creation.	<code>git pull --rebase origin main</code> , then push. This is why step 10 said not to tick those boxes.
refusing to merge unrelated histories	Same cause, worse form.	<code>git pull --allow-unrelated-histories origin main</code> , resolve, push. Or delete the GitHub repository and recreate it empty.
src refspec main does not match any	No commits yet, or the branch has another name.	<code>git log --oneline</code> to check you have commits; <code>git branch -M main</code> .
File rejected as too large	A model file or dataset slipped past <code>.gitignore</code> .	Remove it from history, add the pattern to <code>.gitignore</code> , re-check with <code>git ls-files</code> .
I pushed <code>.env</code>	The ignore rule was wrong.	Revoke every key in it immediately (step 13), then clean history. Revoke first.
Everything shows as modified after cloning elsewhere	Line endings.	The <code>core.autocrlf</code> setting from step 09.

DO THIS NOW

1. Run `git ls-files` and read every line aloud. Confirm no `.env`, no `.venv`, no weights.
2. Run the grep from Listing 18.1 and confirm it says “clean”.
3. Add the remote, confirm it with `git remote -v`, and push.
4. Open the repository in a browser and read your own README as a stranger would.
5. Make one small improvement, then commit and push it — practising the four-command loop.

Verify — your repository is on GitHub, its README renders, and it contains no secret and no gigabyte.

INSTRUCTOR NOTE — WHERE PEOPLE GET STUCK

The rejected first push, caused by a README created on GitHub in step 10 — expect a handful, and have the `--rebase` line ready. The genuinely important habit here is `git ls-files` before the first push; make everyone read their list out loud. Someone will have `.env` in theirs, and catching it thirty seconds before publication is the single most valuable moment in the whole document.

A APPENDIX

The day-one script

Steps 02 through 06 in one file you can run once and reread later. It is safe to run twice — every step checks whether the work is already done and says so rather than repeating it. It installs nothing outside your home folder, and it never writes a secret.

Read it before you run it. It is sixty lines, and reading a setup script before executing it is a habit worth more than the twenty minutes this one saves you.

The macOS version: it uses Homebrew if you have it and falls back to `uv` if you do not, so it works with or without an administrator password.

```
#!/usr/bin/env bash
# Day-one setup for llm-starter. Safe to run twice.
set -euo pipefail

PROJECT="${1:-$HOME/projects/llm-starter}"
RC="$HOME/.zshrc"

say() { printf '\n== %s\n' "$1"; }
ok() { printf ' [ok] %s\n' "$1"; }
skip() { printf ' [skip] %s\n' "$1"; }
have() { command -v "$1" >/dev/null 2>&1; }

say "1. machine"
printf ' arch=%s shell=%s\n' "$(uname -m)" "${SHELL:-?}"
df -h "$HOME" | tail -1 | awk '{print " disk: \"$4\" free"}'
xcode-select -p >/dev/null 2>&1 && ok "command line tools present" || echo " run:
xcode-select --install"

say "2. ~/.local/bin on PATH"
mkdir -p "$HOME/.local/bin"
case ":{PATH}:" in
  *:$HOME/.local/bin:*) skip "already on PATH" ;;
  *) echo 'export PATH="$HOME/.local/bin:$PATH"' >> "$RC"
    export PATH="$HOME/.local/bin:$PATH"
    ok "added to $RC — open a new terminal afterwards" ;;
esac

say "3. python 3.11"
if have python3.11; then skip "python3.11 present"
elif have brew; then brew install python@3.11 && ok "installed via Homebrew"
else
  curl -LsSf https://astral.sh/uv/install.sh | sh
```



```

export PATH="$HOME/.local/bin:$PATH"
uv python install 3.11 && ok "installed via uv – no admin needed"
fi

say "4. git identity"
if have git; then ok "git present"
else echo "    git missing – install it, then rerun"; exit 1; fi
if [ -z "$(git config --global user.name || true)" ]; then
    printf '    name for commits: '; read -r N; git config --global user.name "$N"
    printf '    email for commits: '; read -r E; git config --global user.email "$E"
    ok "identity set"
else skip "identity already set"; fi
git config --global init.defaultBranch main

say "5. ssh key"
if [ -f "$HOME/.ssh/id_ed25519" ]; then skip "key exists – never overwriting it"
else ssh-keygen -t ed25519 -C "$(git config --global user.email)" -f
"$HOME/.ssh/id_ed25519"; fi
printf '\n    Add this PUBLIC key to github.com → Settings → SSH keys:\n\n'
sed 's/^/    /' "$HOME/.ssh/id_ed25519.pub"

say "6. project and environment"
mkdir -p "$PROJECT"; cd "$PROJECT"
if [ -d .venv ]; then skip ".venv exists"
else python3.11 -m venv .venv && ok ".venv created"; fi
source .venv/bin/activate
python -m pip install --quiet --upgrade pip
python -m pip install --quiet openai python-dotenv
ok "openai and python-dotenv installed"

say "7. the files that must exist before the first commit"
[ -f .gitignore ] || printf
'.venv/\n__pycache__/\n*.pyc\n.env\n.env.*\n!.env.example\nmodels/\n*.safetensors\n'
> .gitignore
[ -f .env.example ] || printf
'OPENAI_API_KEY=\nANTHROPIC_API_KEY=\nGOOGLE_API_KEY=\nHF_TOKEN=\n' > .env.example
[ -f .python-version ] || echo "3.11" > .python-version
[ -d .git ] || git init -q
ok "gitignore, env template, python-version, repository"

say "8. verify"
printf '    python      %s\n' "$(python -V)"
printf '    interpreter %s\n' "$(python -c 'import sys;print(sys.executable)')"
printf '    project      %s\n' "$PROJECT"

echo
echo "    Next:"

```

```
echo "    1. Open a NEW terminal so the PATH change applies."
echo "    2. Add the public key above to GitHub, then: ssh -T git@github.com"
echo "    3. cp .env.example .env — then put your real keys in it (step 13)."
echo "    4. Continue from step 08: VS Code."
echo
```

Listing A.1 — Save as `setup.sh`, then run `bash setup.sh`. The numbered sections map onto steps 02 to 06.

What it deliberately does not do: install Docker, touch a driver, or create your `.env`. The first two need administrator rights and carry real consequences; the third is a decision you should make with your eyes open, in step 13.

B APPENDIX

Printable checklist

One line per proof, with what you should see. Print this page and tick in pencil. The number on the right is the step that fixes a failure.

RUN THIS	YOU SHOULD SEE	STEP
☐ <code>pwd</code>	Your current folder — and you know how to change it	01
☐ <code>echo \$PATH tr ':' '\n'</code>	<code>~/local/bin</code> present, no duplicates	02
☐ <code>which -a python3</code>	You can name every interpreter listed	02
☐ <code>uname -m</code>	<code>arm64</code>	03
☐ <code>xcode-select -p</code>	A path, not an error	03
☐ <code>python3.11 --version</code>	A 3.11.x version	04
☐ <code>which python (activated)</code>	A path inside <code>llm-starter/.venv</code>	05
☐ <code>python -m pip check</code>	No broken dependencies reported	06
☐ <code>python -m pip list</code>	<code>openai</code> and <code>python-dotenv</code> present	06
☐ <code>code --version</code>	And the status bar names your <code>.venv</code> interpreter	08
☐ <code>git config --global --list</code>	Your name and email are set	09
☐ <code>git status</code>	No <code>.venv</code> and no <code>.env</code> listed	09
☐ <code>ssh -T git@github.com</code>	"Hi <your-username>!"	10
☐ <code>hf auth whoami</code>	Your Hugging Face username	11
☐ <code>df -h .</code>	At least 10 GB free before the model download	11
☐ <code>python check_keys.py</code>	Four characters, and nothing more	13
☐ <code>python chat_openai.py</code>	A streamed reply; <code>/reset</code> clears the memory	14
☐ <code>python chat_local.py</code>	A reply with the wifi switched off	15
☐ <code>rm -rf .venv, then rebuild</code>	A working chat program from committed files only	16
☐ <code>docker run --rm -it --env-file .env llm-starter:0.1</code>	A conversation inside a container	17
☐ <code>git ls-files</code>	About ten files. No <code>.env</code> , no weights.	18
☐ <code>git push</code>	Your README rendering on github.com	18

Name _____ Date _____

All ticked = ready

C

APPENDIX

Glossary

Every term this document uses that someone might not know, in the plainest words that are still true.

TERM	MEANS
API	A way for your program to ask another company's program to do something, over the internet. You send text, you get text back, you are charged for both.
API key	A password identifying your account to an API. Leaking one means someone else spending your money.
Branch	A parallel line of commits in git, so unfinished work does not disturb <code>main</code> .
Commit	A snapshot of your whole project at a moment, with a message saying why.
Container	A running instance of an image: an isolated box with its own filesystem, sharing your machine's kernel.
Dependency	A package your code needs. A <i>transitive</i> dependency is a package your dependency needs.
Docker image	A packaged filesystem plus a default command — your code and everything it needs, frozen together.
Environment variable	A named value the shell hands to the programs it starts. Where secrets belong.
Flag	Extra instruction given to a command. <code>-r</code> and <code>--rm</code> are flags.
Freeze / pin	Recording the exact version of every package, so a rebuild produces the same thing.
Git	A tool that records the history of a folder. Local to your machine.
GitHub	A website that hosts git repositories. Git works perfectly well without it.
Hugging Face	The main public home for open model weights, and the library that downloads them.
Inference	Using a trained model to produce output — what both your scripts do. Training is the other thing, and needs far more hardware.
Instruct model	A model further trained to follow instructions and hold a conversation, rather than merely continue text.
Interpreter	The <code>python</code> program itself, which reads your file and runs it.

TERM	MEANS
Package	Someone else's code that you install and use. <code>openai</code> and <code>torch</code> are packages.
Parameter	One of the numbers inside a model. "1.5B" means one and a half billion of them.
PATH	The list of folders your shell searches when you type a command name. Searched left to right.
pip	The tool that installs Python packages.
Prompt	Two meanings: the text your shell prints while waiting for you, and the text you send a model.
Pull request	A proposal on GitHub to merge one branch into another, so someone can read it first.
Quantised	A model stored with fewer bits per parameter — smaller and faster, slightly worse.
Rate limit	A cap on how often you may call an API. Exceeding it returns <code>429</code> .
Repository	A folder whose history git is tracking.
Shell	The program interpreting what you type in a terminal. Yours is <code>zsh</code> .
SSH key	A pair of files proving your identity to a remote machine. The private half never moves.
Streaming	Receiving a model's reply piece by piece as it is generated, rather than all at once.
System prompt	Standing instructions given to a model before the conversation starts.
Terminal	The window. The shell is the program running inside it.
Token	A chunk of text a model reads and writes — roughly three-quarters of an English word. What you are billed for.
Virtual environment	A folder holding its own Python and its own packages, belonging to one project.
Volume	Storage attached to a container that survives the container being deleted.
Weights	The file containing a model's parameters. Gigabytes. Never in git.
Homebrew	The package manager most Mac developers use to install command-line software.

TERM	MEANS
mps	Apple's interface to the GPU built into your chip. What makes local models usable on a Mac.

WHAT AGES FASTEST IN THIS DOCUMENT

Model names, prices, exact package versions, and the shape of every website you signed up on — those will drift within months, so check the provider's own documentation rather than this page. What will not age: the shell and PATH material, the virtual environment model, the ordering of git before secrets, the leak procedure, the retry rules by status code, and the delete-and-rebuild test. Those have been true for a long time and will outlast several versions of everything else here.

You are done.

REVISION 1.0

Eighteen steps, eighteen rules, and one small project taken from an empty folder to a containerised, version-controlled program that talks to a language model two different ways. Keep this document near the machine, and write in the margins — your own machine's quirks are worth more to you than anything printed here.